



Desafios para os Javanautas

Desafio 1: Melhorias em um Projeto Existente

Instruções:

1. **Baixe o projeto:** Faça o download do zip do projeto `comunicacao_api`.

Documente os endpoints: No arquivo `README.md` da aplicação, identifique e documente a responsabilidade de cada endpoint.

Exemplo simples de documentação:

Endpoints da Aplicação

- **GET /api/usuarios**: Retorna a lista de usuários cadastrados.
- **POST /api/usuarios**: Cadastra um novo usuário.

O dto deve ser preenchido com os seguintes dados: `{ "nome": "", "senha": "" }`

- **PUT /api/usuarios/{id}**: Atualiza as informações de um usuário existente.
- **DELETE /api/usuarios/{id}**: Remove um usuário pelo ID.

2. **Implemente o Swagger:** Use a biblioteca OpenAPI para documentar os endpoints diretamente no código.
3. **Integração com API externa:** Conecte a API de notificação por email (criada durante o curso) para envio de emails de comunicação fazendo as adaptações necessárias no projeto `comunicacao_api` para informações sobre o que já foi comunicado. Se necessário crie novos endpoints.
4. **Melhore a conversão DTO-Entity:** Utilize o MapStruct para gerenciar a conversão entre DTOs e entidades.
5. **Global Exception Handler:** Implemente um `GlobalExceptionHandler` para padronizar e melhorar os retornos de erros. Se necessário crie novas classes de exceção e melhore as mensagens de retorno.
6. **Suba a aplicação no Docker:** Configure o Docker para executar a aplicação.
7. **Testes Unitários:** Desenvolva testes unitários para as camadas Controller e Service.

Desafio 2: API para Mensagens SMS

Objetivo: Implemente uma aplicação para gerenciar o status de mensagens SMS e criar relatórios.

Especificações Técnicas:

1. **API REST:**

- **Atualizar Status:** Crie uma rota para atualizar o status de envio de mensagens SMS no banco de dados.
- **Relatórios:** Implemente uma rota que retorne as mensagens com um status específico das últimas 24 horas.

2. Regras de Negócio:

- IDs devem ser puramente numéricos.
- Status válidos: "ENVIADO", "RECEBIDO", "ERRO DE ENVIO".
- Retorne erros adequados caso o ID não exista no banco.

3. Banco de Dados:

Modelo de exemplo:

```
public class SmsMessage {
    private Long id;
    private String phoneNumber;
    private String status; // ENVIADO, RECEBIDO, ERRO DE ENVIO
    private LocalDateTime sentAt;
}
```

4. Validações:

- Requisições inválidas devem ser rejeitadas.
- Atualizações de status devem ser precisas e consistentes.

Desafio 3: Gerenciamento de Autores e Obras

Construa uma api para gerenciamento de autores e obras. Implemente um CRUD para cadastro de obras relacionados aos autores e também para cadastro de autores.

Requisitos:

- Relacione **Autores** e **Obras** conforme as regras:
 - Um autor pode ter várias obras.
 - Uma obra pode ter vários autores.

Entidades:

1. Autor:

- Nome (obrigatório)
- Sexo
- E-mail (validado, único)
- Data de nascimento (validada)
- País de origem (obrigatório)
- CPF (obrigatório para autores do Brasil, único)

2. Obra:

- Nome (obrigatório)
- Descrição (máximo 240 caracteres)
- Data de publicação ou data de exposição (uma é obrigatória).

Extras:

- Implemente autenticação com JWT.
 - Desenvolva testes unitários para CRUD das entidades.
-

Desafio 4: CRUD com PostgreSQL

Especificações Técnicas:

1. **Banco de Dados:** Configure o PostgreSQL para gerenciar duas entidades: **Produtos** e **Categorias**.
2. **Operações CRUD:**
 - **Produtos:** Nome, descrição, preço, e ID da categoria.
 - **Categorias:** Nome.

Extras:

- Relacione produtos e categorias.
- Utilize Swagger para documentar os endpoints.

Estrutura de Rota Exemplo:

POST /api/produtos
GET /api/produtos/{id}
PUT /api/produtos/{id}
DELETE /api/produtos/{id}



Prazo de Entrega:

Você pode resolver os desafios na sequência que preferir, mas atente-se para as boas práticas e complete os testes antes de submeter sua solução. Boa sorte! 🚀